1. ***Tóm tắt và Phân tích tài liệu***

Bài báo là một bài khảo sát về Kiểm thử Ứng dụng Web (Web Application Testing - WAT) trong giai đoạn từ 2014-2023, dựa trên 314 bài báo khoa học.

Các xu hướng chính:

- Mục tiêu kiểm thử: Tập trung dồn mạnh về bảo mật. Phát hiện lỗ hổng website chiếm tỉ trọng lớn nhất (29.94%), theo sau đó là Kiểm thử bảo mật chung (27.71%).
- Phương pháp sinh ca kiểm thử (Test Case Generation):
    - Model-Based Testing (MBT): Vẫn là phương pháp phổ biến nhất (27.5%), sử dụng các mô hình như UML hoặc biểu đồ trạng thái để sinh test case.
    - AI/Machine Learning: Các phương pháp dựa trên học máy (Learning-based) và thuật toán tối ưu hóa đang nổi lên nhưng chỉ mới chiếm 10%
- Thực thi kiểm thử: Tự động hóa (Automated Execution) chiếm ưu thế tuyệt đối (91.08%). Việc kiểm thử lai (Hybrid) - Kết hợp giữa tự động và thủ công - đang được chú ý để xử lý các kịch bản phức tạp mà máy móc chưa làm tốt

Các vấn đề và thách thức (Current Challenges):

1. Sự “mỏng manh” của Test Case: Các ứng dụng Web Dynamic thay đổi ID, Class hoặc cấu trúc DOM liên tục sau mỗi lần cập nhật code. Điều này khiến các kịch bản kiểm thử (đặc biệt là selenium script) bị hỏng ngay lập tức. Dẫn đến chi phí bảo trì cho bộ test tự động là cực lớn. Dù có dùng AI để tự sửa, nhưng khi UI thay đổi quá lớn, AI cũng bó tay.
2. Hạn chế trong Chuẩn đoán lỗi: Khi một test case thất bại, rất khó để biết nguyên nhân là do lỗi của ứng dụng, do mạng lag hay do test case bị lỗi thời. Các phương pháp Model-based giúp ích một phần, nhưng vẫn gặp vấn đề lớn về False Positives (Báo lỗi giả) và False Negatives (Bỏ sót lỗi). Máy móc chưa đủ thông minh để phân biệt một “tính năng mới” và một “lỗi”.
3. Sự phức tạp của ứng dụng Web hiện đại: Các ứng dụng Web ngày càng phụ thuộc vào nội dung động (Dynamic Content), AJAX và các framework như React/Vue/Angular. Các công cụ kiểm thử truyền thống gặp khó khăn khi xử lý các thay đổi trạng thái không đồng bộ (asynchronous operations)
4. Phân mảnh công cụ (Tool Fragmentation): Có quá nhiều công cụ rời rạc (Selenium cho chức năng, Burp Suite/ZAP cho bảo mật). Việc tính hợp chúng thành 1 quy trình xuyên suốt rất khó khăn.
5. Lỗ hổng Logic (Logic Flaws): Việc chuẩn đoán lỗi thất bại, đặc biệt là các lỗi liên quan đến logic nghiệp vụ(business logic) thay vì lỗi kỹ thuật

(như crash), vẫn là 1 nút thắt cổ chai lớn với tỉ lệ dương tính giả (false positives) cao.

6. Thiếu thước đo chuẩn (Standardized Metrics): Không có bộ tiêu chuẩn thống nhất để đánh giá hiệu quả của các công cụ kiểm thử bảo mật, khiến việc so sánh trở nên khập khiễng.

2. Đề xuất Research Gap trong Penetration Testing

a. Vấn đề từ thực tế (The Problem):

- Công cụ hiện tại: Các công cụ như OWASP ZAP, Burp Suite Scanner, Nessus) hoạt động dựa trên việc khớp mẫu(pattern matching). Rất giỏi phát hiện SQLi, XSS vì các lỗi này có dấu hiệu rõ ràng.
- Điểm mù: Tuy nhiên, tài liệu chỉ ra rằng phát hiện Logic Flaws (Lỗ hổng Logic) là 1 thách thức lớn. Ví dụ : Một người có thể bỏ qua thanh toán để nhận hàng, hoặc sửa ID trong URL để xem dữ liệu của người khác (IDOR - Broken Access Control). Máy quét tự động không hiểu “quy trình mua hàng” (vì mỗi website sẽ có những cách thức khác nhau), nên không biết việc bỏ qua bước thanh toán là sai.

b. Hướng đi mới (The Opportunity):

- Tài liệu gợi ý hướng tương lai là tích hợp LLM (Gemini, GPT) để sinh test case tốt hơn, đặc biệt là cho các biểu mẫu web (web forms) cần thông tin ngữ cảnh.
- **Đồng thời, việc sử dụng Multi-agent reinforcement learning(MARL) cũng được đề cập để tăng độ phủ kiểm thử** **(này chắc nghiên cứu sau, giờ tập trung vào LLM trước)**
- Gap: Chưa có nhiều nghiên cứu hoàn chỉnh giữa LLM Agents (có khả năng tư duy, hiểu ngữ cảnh nghiệp vụ) với quy trình Penetration Testing để tấn công vào các luồng nghiệp vụ phức tạp (multi-step attacks) mà các scanner truyền thống bỏ sót.

3. Tự nghiệm về các Research Gap của bản thân

- Bản thân rất thích các công cụ LLM về khả năng tự nghiệm của các mô hình ngôn ngữ lớn. Và hoàn toàn có thể tận dụng điều này để đọc ngữ nghĩa của các trang web về loại lỗ hổng là Business Logic Flaws (có thể cụ thể hơn là Broken Access Control).
- **Nếu đi theo con đường Business Logic Flaws thuần chủng:**
    + Kiểm thử trong một quy trình có nhiều bước thực thi dưới sự kiểm soát và “cảm nhận” ngữ nghĩa của LLM.

    + Sinh ra các test case dựa trên LLM để bắn thử exploit để kiểm thử về giả định mà LLM đã đưa ra sau khi hiểu được ngữ nghĩa của trang web.
    + Các loại hình bao gồm: Quy trình giao dịch, Bypass các điểm yếu về xác thực (authentication), xử lý các thông số bị sai, …
- **Nếu đi theo con đường Broken Access Control (nên):**
    + Cũng tương tự là sự kiểm soát ngữ nghĩa của LLM để có thể phân biệt được điểm này là có hay không về loại lỗ hổng BAC.
    + Cho LLM hiểu được 1 user thường thực thi và 1 user có role hoặc khác user để so sánh về khả năng thực thi.

4. Các loại lỗ hổng cụ thể cho từng loại BAC hoặc BLF.

- ***Phân loại Business Logic Flaws:***

A. Lỗ hổng liên quan đến Tài chính & Số lượng (Financial & Quantity Logic)

Đây là dạng dễ demo nhất và "sát sườn" nhất với các hệ thống thương mại điện tử.

- **Số lượng âm (Negative Quantity):**
    - *Mô tả:* Kẻ tấn công đặt mua sản phẩm với số lượng `-1`. Hệ thống tính toán: `Giá $100 * (-1) = -$100`. Tổng hóa đơn bị trừ đi $100 thay vì cộng thêm.
    - *Ví dụ:* Mua 1 cái iPhone ($1000) và -2 cái ốp lưng ($50/cái). Tổng tiền: $1000 - $100 = $900.
- **Làm tròn số thập phân (Decimal Rounding Errors):**
    - *Mô tả:* Lợi dụng cách hệ thống làm tròn số tiền.
    - *Ví dụ:* Một token giá $0.99. Mua 100 cái giá $99. Nhưng nếu mua 1 cái nhiều lần, hệ thống làm tròn $0.99 thành $0 (nếu code sai) hoặc tích lũy lỗi làm tròn để trục lợi.
- **Thao túng giá (Price Manipulation):**
    - *Mô tả:* Giá tiền được gửi từ Client lên Server trong gói tin POST và Server tin dùng nó để thanh toán mà không check lại database.
    - *Ví dụ:* Sửa parameter `price=1000` thành `price=1` trong Burp Suite khi bấm thanh toán.

Có 1 lỗi nữa cũng liên quan đến số nhưng mà không hẳn là Financial & Quantity Logic là web dễ bị nhầm lẫn và bug giữa 1 và 1.0 (để tìm lại xem lỗi này tạo lại kiểu gì, hình như Python hay JS gì đó)

B. Bỏ qua quy trình (Workflow Bypass)

- **Nhảy cóc trạng thái (Skipping Steps):**
  - *Mô tả:* Quy trình là A -> B -> C. Hacker gọi trực tiếp API của bước C mà không qua B.
  - *Ví dụ:* Quy trình: `Chọn hàng` -> `Thanh toán` -> `Xác nhận đơn`. Hacker bắt request của bước `Xác nhận đơn` và gửi luôn, bỏ qua bước `Thanh toán`.
- **Bỏ qua xác thực 2 bước (2FA Bypass):**
  - *Mô tả:* Hệ thống yêu cầu nhập OTP. Hacker gọi trực tiếp API `/api/home` hoặc `/api/admin` mà không gửi kèm token xác thực OTP, nhưng server lại quên check ở các endpoint đó.

C. Lạm dụng logic giới hạn (Limit & Constraint Logic)

- **Race Condition (Điều kiện đua):**
  - *Mô tả:* Thực hiện nhiều hành động song song cùng lúc để vượt qua giới hạn kiểm tra.
  - *Ví dụ:* Bạn có mã giảm giá chỉ dùng 1 lần. Bạn dùng tool gửi 10 request "Áp dụng mã" cùng lúc trong 1 mili-giây. Server xử lý đa luồng, chưa kịp cập nhật trạng thái "đã dùng" thì cả 10 request đều thành công.
- **Inventory Depletion (Phá kho):**
  - *Mô tả:* Thêm sản phẩm vào giỏ hàng nhưng không mua, giữ nó ở đó khiến người khác không mua được (Denial of Service về mặt nghiệp vụ).
- ***Phân loại Broken Access Control:***

A. Leo thang đặc quyền dọc (Vertical Privilege Escalation)

Từ user thường lên admin/mod.

- **Truy cập URL không được bảo vệ (Unprotected URL Access):**
  - *Mô tả:* Admin panel bị ẩn khỏi menu nhưng URL vẫn truy cập được.
  - *Ví dụ:* User thường (hoặc người ngoài không đăng nhập) gõ trực tiếp `/admin/users` hoặc `/robots.txt` thấy đường dẫn lạ và truy cập vào được.
- **Parameter-based Access Control (Mass Assignment):**
  - *Mô tả:* Server dựa vào tham số client gửi lên để xác định quyền.

    - *Ví dụ:* Khi đăng ký hoặc cập nhật profile, user gửi thêm JSON: `{"username": "hacker", "role": "admin"}`. Server tự động map field `role` vào database mà không lọc.
- **Platform Misconfiguration:**
    - *Mô tả:* Cấu hình sai trên server (Nginx, Apache) hoặc Cloud (AWS S3 bucket public).
    - *Ví dụ:* Truy cập `/admin/` bị chặn, nhưng `/admin` (thiếu dấu /) hoặc `%2fadmin` lại vào được.
    - Có cái bug trên Nginx với Apache Tomcat cũng hay hay á, a coi thử xem (cái này chắc nằm trong Security Misconfiguration rồi nên bỏ qua)

B. Leo thang đặc quyền ngang (Horizontal Privilege Escalation) - IDOR

Từ user A truy cập dữ liệu user B (cùng cấp độ).

- **IDOR (Insecure Direct Object References) (đáng lưu ý nhất vì dễ khai thác):**
    - *Mô tả:* ID của đối tượng (file, user, đơn hàng) bị lộ trên URL hoặc Body và có thể đoán được.
    - *Ví dụ:*
        - URL: `example.com/invoice?id=1001` (Của mình).
        - Hacker sửa thành: `example.com/invoice?id=1002` (Xem của người khác).
- **IDOR với GUID/UUID:**
    - *Mô tả:* Dù ID ngẫu nhiên dài (UUID), nhưng đôi khi nó bị lộ ở nơi khác (ví dụ: trong API `GET /users/me` trả về list friend kèm UUID của họ). Hacker dùng UUID đó để truy cập thông tin kín.

C. Context-dependent Access Control (Sai ngữ cảnh)

- **Business Constraint Bypass:**
    - *Mô tả:* User có quyền truy cập chức năng, nhưng sai thời điểm hoặc sai trạng thái.
    - *Ví dụ:* User được phép "Sửa đơn hàng", nhưng chỉ khi đơn hàng ở trạng thái "Mới tạo". Hacker bắt request sửa đơn hàng và gửi lại khi đơn hàng đã ở trạng thái "Đang giao".

5. Xây dựng workflow của công cụ có thể (Đăng đề xuất)

- Về mô hình kiểm thử sẽ thiên về cả blackbox và whitebox.

- Tự train cho một LLM có khả năng hiểu về các cách để exploit một chương trình tập trung vào BAC/BLF. LLM đó sẽ được train dựa trên một model có thể train trên kaggle.
- Sau quá trình train thì LLM của ta có thể chạy trên local, tự động đưa ra các testcase theo dạng là các file exploit dạng python hoặc các giá trị của tham số để đưa vào “máy exploit” để thử khai thác về loại lỗ hổng đó.
- Có thể train một LLM khác có khả năng nhận diện các traffic từ đó lọc ra các traffic cần test. Ví dụ như là các endpoint cần thiết hoặc quan trọng, tránh các endpoint lạ bên ngoài để tập trung tối đa vào sản phẩm. (Cái này thường dùng cho test blackbox khi chưa có tài liệu các endpoint như file swagger hoặc các tài liệu khác).
- Các bước thực hiện của từng module là:
    + Bước 1: Crawl/Tự sử dụng trang web/Nạp vào danh sách các API endpoint -> Mục tiêu để LLM hiểu được các trang web có những traffic gì, endpoint API gì hay ho có thể có loại lỗ hổng BLF/BAC. Sau đó đưa toàn bộ cho LLM_Filter để lọc các traffic rác hoặc không có tác dụng trong kiểm thử (ví dụ như các file ảnh, js, các nội dung khác mà không có tác dụng - định nghĩa sau).
    + Bước 2: Đưa các traffic đã crawl/tài liệu vào LLM_Exploit để cho LLM hiểu được ngữ cảnh ta đang có các endpoint API gì? Từ đó nó sẽ sinh ra các test case để khai thác. Có thể là các file python rồi tự động chạy terminal để kiểm thử luôn.
    + Bước 3: Quá trình exploit sẽ được diễn ra. Từng loại exploit sẽ nằm trong từng thư mục để mà có thể phân loại và có đánh giá cuối cùng.
    + Bước 4: Cuối cùng, ta cần LLM (có thể là loại Filter) để gom PoC có thể khai thác được và đưa ra một báo cáo cuối cùng là nó đang gặp loại lỗ hổng gì. Từ đó đưa ra đánh giá cuối cùng.
- Về thành phần nâng cao, ta có thể sử dụng Multi Agent để nó có thể thao tác, trao đổi với nhau để nó có thể hoạt động tốt hơn.

2, Execute Agent

- KLTN: Nó bắn CVE/PoC lên mục tiêu và nhận thử kết quả.
- ĐT: Sinh testcase -> Bắn vào web để xem kết quả.

3,

- Sử dụng LLM Filter để chọn API đáng để tấn công dựa trên điểm số, tiêu chí

Trong đề tài bài báo mới những điểm có thể bổ ích cho đồ án chuyên ngành
3. PSM (Penetration testing State Machine) là gì?
Đây chính là "linh hồn" và điểm sáng tạo nhất của bài báo này. PSM (Máy trạng thái Kiểm thử Xâm nhập) là một kiến trúc được lấy cảm hứng từ mô hình toán học FSM (Máy trạng thái hữu hạn - Finite State Machine).
Thay vì gom tất cả lịch sử chat và bắt LLM tự suy nghĩ từ đầu đến cuối (dễ làm LLM bị tràn bộ nhớ, ảo giác, hoặc kẹt trong vòng lặp vô tận), PSM chia nhỏ toàn bộ quá trình pentest thành các "Trạng thái" (States) độc lập. PSM bao gồm 2 loại trạng thái luân phiên nhau:
A. Agent States (Trạng thái Tác tử - Nơi LLM làm việc): Gồm 3 trạng thái: Scanning (Quét), Reconnaissance (Trinh sát), và Exploitation (Khai thác).

- Tại đây, LLM được cấp các công cụ (như Terminal, trình duyệt Playwright, công cụ Search) để tương tác với mục tiêu.
- Đặc biệt, Agent State chỉ nhận dữ liệu đầu vào từ trạng thái ngay trước đó, chứ không phải toàn bộ lịch sử hội thoại.

B. Rule States (Trạng thái Quy tắc - Nơi code cứng kiểm soát LLM): Gồm 2 trạng thái: Selection (Lựa chọn lỗ hổng) và Check (Kiểm tra kết quả).

- Tại đây, không sử dụng LLM. Thay vào đó, nó dùng các bộ quy tắc (rules) được lập trình sẵn để lọc, làm sạch dữ liệu đầu ra của Agent, loại bỏ rác, và đánh giá xem bước vừa rồi có thành công không.
- Ví dụ: Nếu trạng thái Khai thác (Agent) trả về một đống log báo lỗi, trạng thái Check (Rule) sẽ dùng code parse log đó, nếu thấy vượt quá số lần thử cho phép, nó sẽ bắt hệ thống "nhảy" về trạng thái Selection để chọn CVE khác thay vì để LLM tiếp tục loay hoay sửa lỗi.

Tại sao Mindset này lại hay?

- Giải phóng bộ nhớ (Context Limit): Bằng cách chặn sự phụ thuộc vào lịch sử dài, mỗi trạng thái chỉ xử lý đúng ngữ cảnh của nó, giúp tiết kiệm token và giữ cho LLM luôn "tỉnh táo".
- Cơ chế chống kẹt (Graceful Recovery): Nếu LLM bị ảo giác sinh ra lệnh sai liên tục, hệ thống sẽ ép buộc thoát khỏi trạng thái đó (force exit), xóa bỏ ngữ cảnh lỗi, và quay lui (backtrack) về trạng thái trước đó để

thử con đường khác. Điều này giúp toàn bộ hệ thống không bao giờ bị "crash" ngang.

-

Sau khi phân tích đề tài của khóa luận để matching và dễ triển khai hơn với một sinh viên ATTT. Thì bây giờ ta không thu thập dataset để training nữa. Mà ta sẽ thu thập RAG? (Cần tìm hiểu lại cái này) để cho LLM có thể có góc nhìn đa dạng hơn về các testcase để thực hiện sinh testcase.

- Kiến trúc cơ bản hiện tại:

Crawll trang web -> Cho LLM_Filter để lọc ra các APIs đáng để pentest -> Ném cho LLM_Exploit để sinh testcase dựa trên API đó và Execute, sinh lại kịch bản -> Kết quả của API đó -> Vòng lặp tiếp cho các API khác cho đến hết API cần test.

- Kiến trúc nâng cao:

Crawl trang web -> Cho LLM_FIlter có khả năng Crawl (lọc và hiểu ngữ cảnh API) để lọc ra APIs đáng pentest -> LLM_Exploit (có 2 con để cho chúng nó giao tiếp và thống nhất với nhau 1 exploit và có thể là phân chia công việc để tìm kiếm thêm thông tin) nhận được và sinh testcase, Execute, xin thêm ngữ cảnh từ LLM_Filter để đánh giá kết quả, nếu sai hoặc không tấn công được sẽ sinh lại kịch bản/testcase mới (2 con LLM sẽ có thể hỏi nhau liên tục?) -> Trả về kết quả -> Vòng lặp cho đến khi test xong.

GET /api/user?uid=abcd + cookie -> Response -> uid, name, birthday, …

- LLM_Exploit sẽ hỏi LLM_3 GET /api/user?uid=xyzt + cookie (abcd) => Nó phản hồi gì? Để viết exploit đánh giá chuẩn nhất.

=> Để chứng minh là mình có test và chứng minh PoC, …

1. Kiến trúc luồng dữ liệu tuyến tính (Linear Pipeline) vs. Kiến trúc tương tác đa tác tử (Multi-Agent Collaboration)

- Điểm tồn đọng trong kiến trúc Khóa luận: Khóa luận tổ chức các tác tử chạy theo một luồng tuyến tính một chiều: Recon -> Planning -> Execution. Dữ liệu giao tiếp là các file JSON tĩnh (như recon_artifact.json, plan.json). Thiết kế này khiến Tác tử Thực thi (Execution Agent) bị thụ động hoàn toàn. Nó chỉ nhận plan.json và chạy lệnh, không tham gia tìm kiếm hay đánh giá,. Nếu chạy lỗi, nó chỉ biết thử lại cơ bản hoặc dừng, chứ không thể tự "suy nghĩ" để viết lại kịch bản.
- Triển khai trong Đồ án của bạn: Thay vì truyền file JSON cứng nhắc một chiều, kiến trúc của bạn đề xuất sử dụng Multi-Agent tương tác qua lại liên tục. Cụ thể, mô-đun LLM_Exploit (sau khi sinh test case và chạy) nếu gặp thất bại, có thể giao tiếp ngược lại với LLM_Filter để xin thêm ngữ cảnh API, hoặc tự điều chỉnh file Python ngay trong runtime. Sự linh hoạt trong giao tiếp giữa các mô-đun này giải quyết được điểm yếu "giao tiếp một chiều" của khóa luận.

2. Tồn đọng về Kiến trúc thực thi: Sự phụ thuộc vào mã tĩnh (Static PoC) vs. Sinh mã động (Dynamic Testcase Generation)

- Điểm khuyết của khóa luận: Về mặt kiến trúc, hệ thống được thiết kế phụ thuộc hoàn toàn vào các mã khai thác của bên thứ ba. Nhóm tác giả thừa nhận ở phần Hướng phát triển rằng nút thắt lớn nhất là "hiện tượng PoC tồn tại nhưng khó chạy vẫn phổ biến do phụ thuộc môi trường, phiên bản interpreter, toolchain". Tức là kiến trúc tải PoC từ ExploitDB/GitHub về chạy rất dễ "gãy" (crash) vì môi trường chạy thực tế không khớp với môi trường viết code ban đầu.
- Triển khai trong đề tài của bạn: Thay vì xây dựng một mô-đun đi tải code của người khác về chạy, kiến trúc của bạn sử dụng API LLM (LLM_Exploit) để tự sinh mã khai thác (Python scripts) ngay tại runtime (Zero-shot generation) dựa trên payload thực tế. Kiến trúc sinh mã động này giải quyết triệt để sự phụ thuộc vào các thư viện phần mềm hay phiên bản hệ điều hành cứng nhắc, làm cho quá trình Execution linh hoạt và ít lỗi môi trường hơn hẳn.

3. Cơ chế Tự sửa lỗi và Sinh lại kịch bản (Generative Debugging Loop)

- Tồn đọng của khóa luận: Khi lệnh khai thác thất bại, khóa luận dùng cơ chế truy hồi (RAG) để sửa các tham số của lệnh (đổi cờ flag, đổi phiên bản, đổi mục tiêu tấn công) rồi chạy lại. Bản chất và nguyên lý trong PoC/CVE không thay đổi.
- Nội dung cần thực hiện: Đối với BLF/BAC, nếu một hướng bypass thất bại, bạn cần xây dựng vòng lặp Generative Debugging. Khi nhận HTTP 403 (Forbidden) hoặc 400 (Bad Request), LLM_Exploit không chỉ sửa tham số, mà phải viết lại toàn bộ luồng logic của file testcase. Ví dụ: Lần 1 thử truyền price=-1 thất bại -> Vòng lặp đưa response lỗi cho LLM -> Lần 2 LLM tự viết lại kịch bản đổi thành truyền array price=[-1] hoặc khai thác lỗi làm tròn số.

* Công việc:

- Soạn ra playbooks cho BAC/BLF thông qua crawl portswigger, các PoC của nền tảng Bug Bounty, … => Mục đích tạo RAG cho LLM_Exploit và Filter.
- Có model sẵn rồi ta dùng MCP/Langchain để model thao tác các lệnh/file, tương tác qua lại với nhau giữa các LLM (1 Filter, 2 Exploit).
- Nguồn để pentest -> 1 Portswigger, 2 là tự dựng lên (vibe code).

- Crawl nên dùng LLM_Filter để crawl. Playwright -> Có khả năng bắt được các API/giao diện web thô, … => Dùng chính con này sau khi crawl ra ta sẽ lọc các API đáng pentest. (Dễ)
- Đưa các cặp APIs (Req/Res) vào LLM_Exploit (2 con) để nó tương tác với nhau để viết exploit => Thực thi exploit và sửa kịch bản nếu sai.
- Trả về kết quả vào file .md là thao tác với testcase này thì có khai thác được không? Cho đến khi hết danh sách/người dùng dừng lại.
- Cái khó/tốn thời gian nhiều: Các LLM giao tiếp với nhau để đưa ra quyết định, Xây playbooks, Nguồn để tấn công